

La Cairo University

Faculty of Computers and Artificial Intelligence



Database Systems-1

Phase 2
Project: Food Ordering System Project

Report Phase 2

Contents

-Cover Sheet	1
-Code	2

1-Cover Sheet:

Team :

ID	Name
20235040	Mariam Ahmed Mohamed Ismail
20236101	MennaTullah Hisham Albeshry
20236024	Ganna Mohamed Abdelrahman
20235021	Omar Ashraf Mohamed
20235053	Mohamed Ahmed Mohamed
20236096	Ismail Emad Eldin Mohamed

Section: S5

TA: Nour Ayman

Project: Food Ordering System Project

Program: Special Zayed

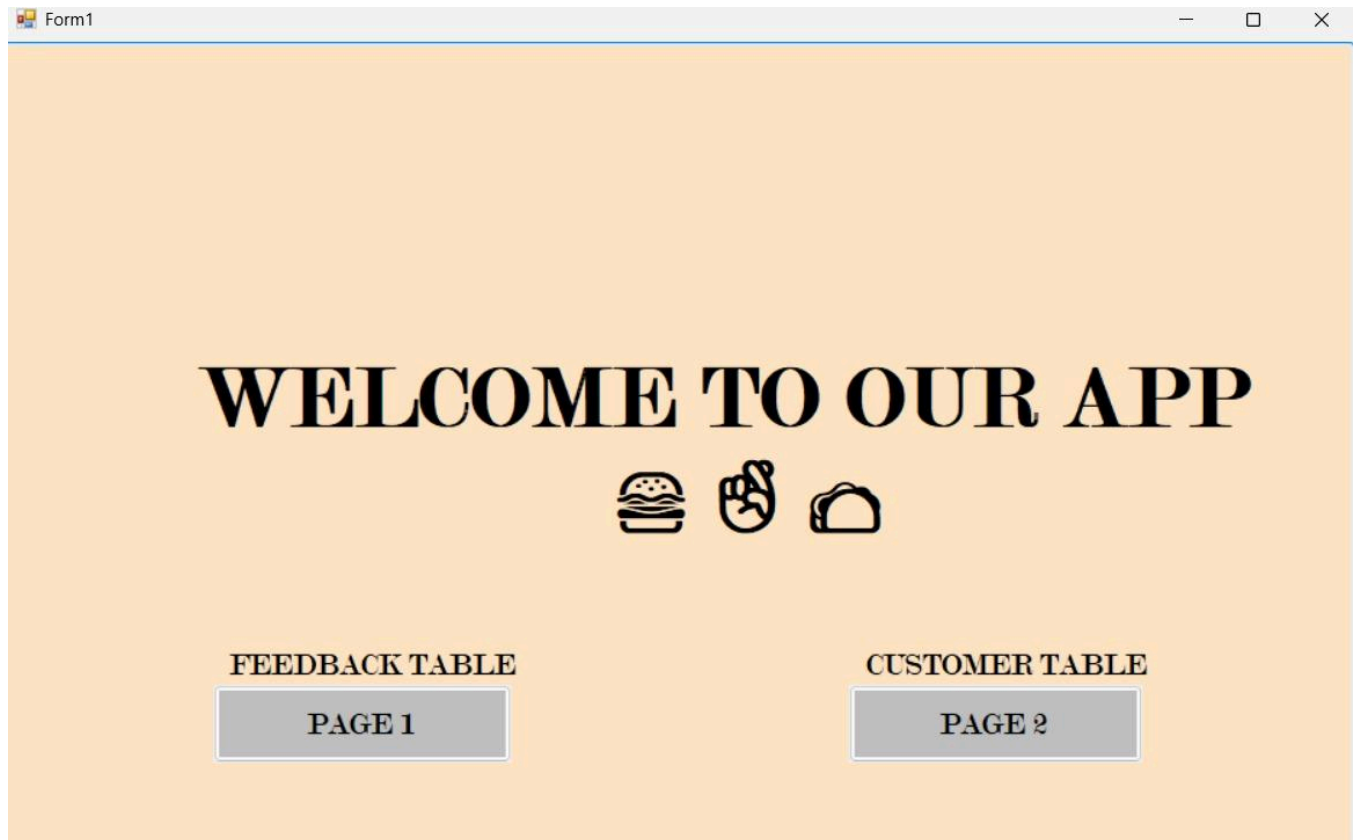
Purpose:

The purpose of this food ordering system is to streamline the process of ordering food for customers while enhancing operational efficiency for restaurants. By providing a user-friendly platform, the system allows customers to browse menus, customize orders, and make payments seamlessly, reducing wait times and improving convenience. For restaurants, it automates order management, minimizes errors, and optimizes kitchen workflows, ultimately boosting customer satisfaction and business productivity.

The food ordering system includes features to identify customers' favorite dishes and seasonal food preferences, while allowing users to add, edit, or remove their feedback. It efficiently manages daily customer data and sales records, and simplifies the billing process, enhancing overall operational effectiveness.

This report explains and provides all the functions and steps used in the implementation of this system.

C# Code:



First of all the design:-

- 1) This is the main form that pops up once the app is opened
- 2) It has two buttons

2.1) the first button “page 1” takes the user to the feedback table by running the code here

```
private void button2_Click(object sender, EventArgs e)
{
    Form2 form2 = new Form2();
    form2.Show();
}
```

Which allows the user to access the feedback table form allowing him to use the functions implemented in the form.

2.2) the second button “page 2” takes the user to the customer table by running the code here

```
private void button3_Click(object sender, EventArgs e)
{
    Form3 form3 = new Form3();
    Form3.Show();
}
```

Which allows the user to access the customer table form allowing him to use the functions implemented in the form.

1)Feedback table:

Form2

RATING

COMMENT

EMAIL

MEALID

	MealID	FeedbackID	CID	Rating	Comment
*					

INSERT UPDATE DELETE SHOW DATA

The picture above shows the for that is displayed to the user which will allow him to delete/insert/update the feedback table:

-The delete function works as follows:-

1)The user enters his email and then clicks the button to show data which shows all the feedback related to that email and its info (MealID , FeedbackID , Rating and comment).

2)The user clicks the delete button which takes him to this form

Form5

PLEASE ENTER THE FEEDBACKID THAT YOU WANT TO DELETE

DELETE

3) This form allows the user to delete any feedback related to his email by entering the feedback id taken shown in the data grid view in the previous form and by clicking the button delete which works by running the code below.

```

private void btnDeleteFeedback_Click(object sender, EventArgs e)
{
    // 1. Validate input is provided

    if (string.IsNullOrEmpty(txtFeedbackID.Text))
    {
        MessageBox.Show("Please enter a Feedback ID.", "Input Required", MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    // 2. Validate numeric value

    if (!int.TryParse(txtFeedbackID.Text.Trim(), out int feedbackId) || feedbackId <= 0)
    {
        MessageBox.Show("Feedback ID must be a positive number.", "Invalid Input", MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    try
    {
        using (SqlConnection con = new SqlConnection(@"Data Source=.SQLEXPRESS;Initial Catalog=app;Integrated Security=True"))
        {
            con.Open();

            // 3. Check if Feedback ID exists

            using (SqlCommand checkCmd = new SqlCommand("SELECT COUNT(*) FROM Feedback WHERE FeedbackID = @FeedbackID",
con))
            {
                checkCmd.Parameters.AddWithValue("@FeedbackID", feedbackId);

                int exists = (int)checkCmd.ExecuteScalar();

                if (exists == 0)
                {

```

```
        MessageBox.Show("No feedback found with the given Feedback ID.", "Not Found", MessageBoxButtons.OK, MessageBoxIcon.Information);
```

```
        return;
```

```
    }
```

```
}
```

```
// 4. Confirm deletion
```

```
DialogResult confirm = MessageBox.Show(
```

```
"Are you sure you want to permanently delete this feedback?",
```

```
"Confirm Deletion",
```

```
MessageBoxButtons.YesNo,
```

```
MessageBoxIcon.Warning);
```

```
if (confirm != DialogResult.Yes)
```

```
    return;
```

```
// 5. Perform deletion
```

```
using (SqlCommand deleteCmd = new SqlCommand("DELETE FROM Feedback WHERE FeedbackID = @FeedbackID", con))
```

```
{
```

```
    deleteCmd.Parameters.AddWithValue("@FeedbackID", feedbackId);
```

```
int rowsAffected = deleteCmd.ExecuteNonQuery();
```

```
if (rowsAffected > 0)
```

```
{
```

```
    MessageBox.Show("Feedback deleted successfully!", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);
```

```
    txtFeedbackID.Clear(); }}
```


-The insert function works as follows:

- 1)The user enters his email address and password the code checks if this email address is registered in the database
- 2)If the user is registered the user enters the meal id for the meal that he wants to leave the feedback for
- 3)The user must leave a rating from 1 to 5
- 4)The user can leave a comments if he/she wants to (the user is not obligated to leave a comment) then the user clicks on the insert button

This is done using the following function:

```
private void btninsert_Click(object sender, EventArgs e)
{
    try
    {
        if (string.IsNullOrEmpty(txtEmail.Text) ||
            string.IsNullOrEmpty(txtMealId.Text) ||
            string.IsNullOrEmpty(txtRating.Text))
        {
            MessageBox.Show("All fields are required.", "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
        if (!int.TryParse(txtMealId.Text, out int mealId) || mealId <= 0)
        {
            MessageBox.Show("Please enter a valid Meal ID.", "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
        if (!int.TryParse(txtRating.Text, out int rating) || rating < 1 || rating > 5)
        {
            MessageBox.Show("Please enter a valid rating (1-5).", "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
        using (SqlConnection con = new SqlConnection(connectionString))
```

```

{

    con.Open();

    using (SqlCommand checkMealCmd = new SqlCommand("SELECT COUNT(*) FROM Meal WHERE MealID = @MealID", con))

    {

        checkMealCmd.Parameters.Add("@MealID", SqlDbType.Int).Value = mealId;

        int mealCount = (int)checkMealCmd.ExecuteScalar();

        if (mealCount == 0)

        {

            MessageBox.Show("The Meal ID does not exist. Please enter a valid Meal ID.", "Validation Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);

            return;

        }

    }

    int customerId;

    using (SqlCommand getCustomerCmd = new SqlCommand("SELECT CID FROM CUSTOMER WHERE EMAIL = @Email", con))

    {

        getCustomerCmd.Parameters.Add("@Email", SqlDbType.NVarChar).Value = txtEmail.Text;

        object result = getCustomerCmd.ExecuteScalar();

        if (result == null)

        {

            MessageBox.Show("This email is invalid. Please use a different email.", "Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);

            return;

        }

        customerId = Convert.ToInt32(result);

    }

    using (SqlCommand cmd = new SqlCommand(

        "INSERT INTO FEEDBACK (CID, MealID, Rating, Comment) VALUES (@CID, @MealID, @Rating, @Comment)", con))

    {

        cmd.Parameters.Add("@CID", SqlDbType.Int).Value = customerId;

        cmd.Parameters.Add("@MealID", SqlDbType.Int).Value = mealId;

        cmd.Parameters.Add("@Rating", SqlDbType.Int).Value = rating;

```

```

        cmd.Parameters.Add("@Comment", SqlDbType.NVarChar).Value = txtComment.Text ?? (object)DBNull.Value;

        cmd.ExecuteNonQuery();

        MessageBox.Show("Feedback inserted successfully!", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);

    }

    RefreshTable(customerId);

}

}

catch (Exception ex)

{

    MessageBox.Show($"An unexpected error occurred: {ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

}

}

private void RefreshTable(int customerId)

{

    try

    {

        if (DesignMode) return;

        using (SqlConnection con = new SqlConnection(connectionString))

        {

            con.Open();

            using (SqlDataAdapter da = new SqlDataAdapter(

                "SELECT F.MealID, F.Rating, F.Comment FROM FEEDBACK F WHERE F.CID = @CID", con))

            {

                da.SelectCommand.Parameters.Add("@CID", SqlDbType.Int).Value = customerId;

                DataTable dt = new DataTable();

                da.Fill(dt);

                FeedbackTable.DataSource = dt;

            }

        }

    }

}

```

```

catch (Exception ex)

{

    MessageBox.Show($"An unexpected error occurred while refreshing the table: {ex.Message}", "Error", MessageBoxButtons.OK,
    MessageBoxIcon.Error);

}

}

```

-The update function works as follows:

- 1)The user enters his email address and the code checks if this email address is registered in the database
- 2)If the user is registered the user enters the meal id for the meal that he wants to update its feedback
- 3)The user can update the rating but has to be from 1 to 5
- 4)The user can update their comments.

then the user clicks on the update button .

This is done using the following function:

```

private void btnUpdate_Click(object sender, EventArgs e)

{

    {

        try

        {

            // Validation

            if (string.IsNullOrEmpty(txtEmail.Text) ||

                string.IsNullOrEmpty(txtMealId.Text) ||

                string.IsNullOrEmpty(txtRating.Text))

            {

                MessageBox.Show("All fields are required.", "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

                return;

            }

            if (!int.TryParse(txtMealId.Text, out int mealId) || mealId <= 0)

            {

```

```

        MessageBox.Show("Please enter a valid Meal ID.", "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    if (!int.TryParse(txtRating.Text, out int rating) || rating < 1 || rating > 5)
    {
        MessageBox.Show("Please enter a valid rating (1-5).", "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    using (SqlConnection con = new SqlConnection(@"Data Source=.SQLEXPRESS;Initial Catalog=app;Integrated Security=True"))
    {
        con.Open();

        // Check if MealID exists

        using (SqlCommand checkMealCmd = new SqlCommand("SELECT COUNT(*) FROM Meal WHERE MealID = @MealID", con))
        {
            checkMealCmd.Parameters.Add("@MealID", SqlDbType.Int).Value = mealId;

            int mealCount = (int)checkMealCmd.ExecuteScalar();

            if (mealCount == 0)
            {
                MessageBox.Show("The Meal ID does not exist. Please enter a valid Meal ID.", "Validation Error", MessageBoxButtons.OK,
                MessageBoxIcon.Error);

                return;
            }
        }
    }

    // Get CustomerID from email

    int customerId;

    using (SqlCommand getCustomerCmd = new SqlCommand("SELECT CID FROM CUSTOMER WHERE EMAIL = @Email", con))
    {

```

```

getCustomerCmd.Parameters.Add("@Email", SqlDbType.NVarChar).Value = txtEmail.Text;

object result = getCustomerCmd.ExecuteScalar();

if (result == null)
{
    MessageBox.Show("Invalid email. Please use a registered email.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

    return;
}

customerId = Convert.ToInt32(result);
}

// Update the feedback

using (SqlCommand updateCmd = new SqlCommand(

    "UPDATE FEEDBACK SET Rating = @Rating, Comment = @Comment WHERE CID = @CID AND MealID = @MealID", con))
{
    updateCmd.Parameters.Add("@Rating", SqlDbType.Int).Value = rating;

    updateCmd.Parameters.Add("@Comment", SqlDbType.NVarChar).Value = txtComment.Text ?? (object)DBNull.Value;

    updateCmd.Parameters.Add("@CID", SqlDbType.Int).Value = customerId;

    updateCmd.Parameters.Add("@MealID", SqlDbType.Int).Value = mealId;

    int rowsAffected = updateCmd.ExecuteNonQuery();

    if (rowsAffected > 0)
    {
        MessageBox.Show("Feedback updated successfully!", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);

        // Call the refresh method here

        RefreshTable(customerId);
    }
    else

```

```

        {
            MessageBox.Show("No matching feedback found to update.", "Info", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
}

catch (Exception ex)
{
    MessageBox.Show($"An error occurred while updating feedback: {ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
}

```

-The show data function works as follows:

For the feedback table the show data function is used to only the feedbacks related to the email entered, the user enters his email and then clicks the button and all the feedbacks written with that email is displayed in the data grid view and the button is created to help the user sort and delete his feedbacks as it is explained in the delete function. This is done by running this code:-

```

private void btnShow_Click(object sender, EventArgs e)
{
    try
    {
        if (string.IsNullOrEmpty(txtEmail.Text))
        {
            MessageBox.Show("Please enter an email.", "Input Required", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        using (SqlConnection con = new SqlConnection(@"Data Source=.SQLEXPRESS;Initial Catalog=app;Integrated Security=True"))
        {
            con.Open();

```

```

// Get CID from email

int customerId;

using (SqlCommand getCustomerCmd = new SqlCommand("SELECT CID FROM CUSTOMER WHERE EMAIL = @Email", con))
{
    getCustomerCmd.Parameters.Add("@Email", SqlDbType.NVarChar).Value = txtEmail.Text;

    object result = getCustomerCmd.ExecuteScalar();

    if (result == null)
    {
        MessageBox.Show("Email not found in the database.", "Not Found", MessageBoxButtons.OK,
        MessageBoxIcon.Information);

        return;
    }

    customerId = Convert.ToInt32(result);
}

// Fetch Feedbacks related to CID, excluding CID but including FeedbackID

using (SqlDataAdapter da = new SqlDataAdapter(
    "SELECT FeedbackID, MealID, Rating, Comment FROM FEEDBACK WHERE CID = @CID", con))
{
    da.SelectCommand.Parameters.Add("@CID", SqlDbType.Int).Value = customerId;

    DataTable dt = new DataTable();

    da.Fill(dt);

    dgvFeedback.DataSource = dt;
}

}

}

catch (Exception ex)
{
    MessageBox.Show($"Error loading feedback data: {ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
}

```


}

2- Customer table:

	CID	FirstName	LastName	Email	Password	Government	City	District	Street	PhoneNo
*										

The picture above shows the form that is displayed to the user which will allow him to delete/insert/update the customer table:

-The delete function works as follows:-

- 1) The logic behind this delete function is different from the feedback table because this function deletes based on that the user has the email AND the password to his account to be able to delete his info.
- 2) Thus the user deletes his info by entering his email and password and then clicks the delete button which runs this code:-

```
private void btnDelete_Click(object sender, EventArgs e)
{
    string email = txtEmail.Text.Trim();

    string password = txtPassword.Text.Trim();

    if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(password))
    {
        MessageBox.Show("Please enter both Email and Password.", "Input Required", MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    try
    {

```

```

using (SqlConnection con = new SqlConnection(@"Data Source=.\SQLEXPRESS;Initial Catalog=app;Integrated Security=True"))

{

    con.Open();

    // Step 1: Verify customer exists with matching email and password

    int customerId = -1;

    using (SqlCommand checkCmd = new SqlCommand("SELECT CID FROM Customer WHERE Email = @Email AND Password = @Password", con))

    {

        checkCmd.Parameters.AddWithValue("@Email", email);

        checkCmd.Parameters.AddWithValue("@Password", password);

        object result = checkCmd.ExecuteScalar();

        if (result == null)

        {

            MessageBox.Show("No customer found with the provided email and password.", "Not Found", MessageBoxButtons.OK, MessageBoxIcon.Information);

            return;

        }

        customerId = Convert.ToInt32(result);

    }

    // Step 2: Confirm deletion from user

    DialogResult confirm = MessageBox.Show("Are you sure you want to delete this customer and all related data?", "Confirm Deletion", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);

    if (confirm != DialogResult.Yes)

    {

        return;

    }

    // Step 3: Delete phone numbers first due to foreign key constraint

    using (SqlCommand deletePhonesCmd = new SqlCommand("DELETE FROM CustomerPhone WHERE CID = @CID", con))

    {

        deletePhonesCmd.Parameters.AddWithValue("@CID", customerId);

        deletePhonesCmd.ExecuteNonQuery();

    }

    // Step 4: Delete customer

```

```

using (SqlCommand deleteCustomerCmd = new SqlCommand("DELETE FROM Customer WHERE CID = @CID", con))

{

    deleteCustomerCmd.Parameters.AddWithValue("@CID", customerId);

    deleteCustomerCmd.ExecuteNonQuery();

}

MessageBox.Show("Customer and all related data deleted successfully!", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);

// Optional: Refresh UI

btnShow_Click(null, null);

}

}

catch (Exception ex)

{

    MessageBox.Show("Error while deleting: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

}

}

```

-The insert function works as follows:-

1)The user enters his email address , phone number(s), address, password, first and last name and the code checks if this email address is registered in the database

2)If the user is registered the an error message will appear if not the user the customer details will be stored successfully in customer table and the customer phone number(s) will be stored in customer_phoneno table successfully

This is done using the following function:

```

private void btninsert_Click(object sender, EventArgs e)

{

    try

    {

        using (SqlConnection con = new SqlConnection(connectionString))

        {

```

```
con.Open();

// Step 1: Check if the email already exists

using (SqlCommand checkCmd = new SqlCommand("SELECT COUNT(*) FROM CUSTOMER WHERE EMAIL = @Email", con))

{

    checkCmd.Parameters.AddWithValue("@Email", txtEmail.Text);

    int emailCount = (int)checkCmd.ExecuteScalar();

    if (emailCount > 0)

    {

        MessageBox.Show("This email is already registered. Please use a different email.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;

    }

}

// Step 2: Collect new phone numbers from TextBox controls and validate

List<string> newPhoneNumbers = new List<string>();

if (!string.IsNullOrEmpty(txtPhoneNo1.Text)) newPhoneNumbers.Add(txtPhoneNo1.Text.Trim());

if (!string.IsNullOrEmpty(txtPhoneNo2.Text)) newPhoneNumbers.Add(txtPhoneNo2.Text.Trim());

// Enforce at least one phone number

if (newPhoneNumbers.Count == 0)

{

    MessageBox.Show("A customer must provide at least one phone number. Please enter at least one phone number.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

    return;

}

// Step 3: Validate total phone numbers (at most 2)

if (newPhoneNumbers.Count > 2)

{
```

```
        MessageBox.Show("A customer can have at most 2 phone numbers. Please provide only one or two phone numbers.", "Error",  
        MessageBoxButtons.OK, MessageBoxIcon.Error);
```

```
        return;
```

```
    }
```

```
    // Step 4: Insert the new customer and get the generated CustomerID
```

```
    int customerId;
```

```
    using (SqlCommand cmd = new SqlCommand(
```

```
        "INSERT INTO CUSTOMER (EMAIL, PASSWORD, FIRSTNAME, LASTNAME, GOVERNMENT, CITY, STREET, DISTRICT) " +
```

```
        "OUTPUT INSERTED.CID VALUES (@Email, @Password, @FirstName, @LastName, @Government, @City, @Street, @District)",  
        con))
```

```
    {
```

```
        cmd.Parameters.AddWithValue("@Email", txtEmail.Text);
```

```
        cmd.Parameters.AddWithValue("@Password", txtPassword.Text);
```

```
        cmd.Parameters.AddWithValue("@FirstName", txtFName.Text);
```

```
        cmd.Parameters.AddWithValue("@LastName", txtLName.Text);
```

```
        cmd.Parameters.AddWithValue("@Government", txtGovern.Text);
```

```
        cmd.Parameters.AddWithValue("@City", txtCity.Text);
```

```
        cmd.Parameters.AddWithValue("@Street", txtStreet.Text);
```

```
        cmd.Parameters.AddWithValue("@District", txtDistrict.Text);
```

```
        object result = cmd.ExecuteScalar();
```

```
        customerId = Convert.ToInt32(result);
```

```
        MessageBox.Show("Customer inserted successfully!", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);
```

```
    }
```

```
    // Step 5: Insert new phone numbers
```

```
    using (SqlCommand phoneCmd = new SqlCommand("INSERT INTO CustomerPhone (CID, PhoneNo ) VALUES (@CID, @PhoneNo  
)", con))
```

```
    {
```

```
        phoneCmd.Parameters.AddWithValue("@CID", customerId);
```

```
        phoneCmd.Parameters.Add("@PhoneNo ", SqlDbType.VarChar); // Define the parameter once
```

```

foreach (string phoneNumber in newPhoneNumbers)
{
    try
    {
        phoneCmd.Parameters["@PhoneNo "].Value = phoneNumber;

        phoneCmd.ExecuteNonQuery();
    }

    catch (SQLException ex) when (ex.Number == 2627) // 2627 is the error code for primary key violation
    {
        MessageBox.Show($"The phone number '{phoneNumber}' is already associated with this customer. Please use a different
phone number.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

        continue; // Skip to the next phone number
    }
}

MessageBox.Show("Phone numbers inserted successfully!", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);

// Step 6: Refresh the table to show the customer's data and phone numbers

RefreshTable(txtEmail.Text);

con.Close();
}
}

catch (Exception ex)
{
    MessageBox.Show($"An error occurred: {ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}

private void RefreshTable(string email)

```

```

{
    try
    {
        if (DesignMode) return; // Prevent execution in design mode

        using (SqlConnection con = new SqlConnection(connectionString))
        {
            con.Open();

            // Join CUSTOMER and CUSTOMER_PHONENO tables to display customer details and phone numbers

            using (SqlDataAdapter da = new SqlDataAdapter(

                "SELECT C.*, P.PhoneNo " +

                "FROM CUSTOMER C " +

                "LEFT JOIN CustomerPhone P ON C.CID = P.CID " +

                "WHERE C.EMAIL = @Email", con))
            {
                da.SelectCommand.Parameters.AddWithValue("@Email", email);

                DataTable dt = new DataTable();

                da.Fill(dt);

                CustomerDataGrid.DataSource = dt; // Assuming dataGridView1 is the name of your DataGridView
            }

            con.Close();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"An error occurred while refreshing the table: {ex.Message}", "Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}

```

-The update function works as follows:-

1)The user has to enter his email address and password in order to update any field on information

- 2) The user has to enter his email address only if he wants to update his password.
- 3) The code verifies the email that it is written correctly and then checks if this email address is registered in the database and extracts the data related to that email address and stores it.
- 4) If the user changed his password, a message appears confirming with the user if he is sure he wants to update the password.
- 5) After the user clicks on the update button, a message appears confirming the success of the updating process.
- 6) If an error occurred an error message is shown with the reason.

This is done using the following function:

```
private void btnUpdate_Click(object sender, EventArgs e)

{

    try

    {

        string email = txtEmail.Text.Trim();

        string inputPassword = txtPassword.Text.Trim();

        if (string.IsNullOrEmpty(email))

        {

            MessageBox.Show("Please enter the Email.", "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;

        }

        using (SqlConnection con = new SqlConnection(@"Data Source=.\SQLEXPRESS;Initial Catalog=app;Integrated Security=True"))

        {

            con.Open();

            // Try to find customer using just the email

            SqlCommand getCmd = new SqlCommand(

                "SELECT c.*, " +

                "MIN(p.PhoneNo) AS Phone1, " +

                "MAX(p.PhoneNo) AS Phone2 " +
```



```

"FROM Customer c " +

"LEFT JOIN CustomerPhone p ON c.CID = p.CID " +

"WHERE c.Email = @Email " +

"GROUP BY c.CID, c.FirstName, c.LastName, c.Password, c.Government, c.City, c.Street, c.District, c.Email", con);

getCmd.Parameters.AddWithValue("@Email", email);

SqlDataAdapter adapter = new SqlDataAdapter(getCmd);

DataTable dt = new DataTable();

adapter.Fill(dt);

if (dt.Rows.Count == 0)

{

    MessageBox.Show("No customer found with this email.", "Not Found", MessageBoxButtons.OK, MessageBoxIcon.Error);

    return;

}

DataRow original = dt.Rows[0];

int customerId = Convert.ToInt32(original["CID"]);

string originalPassword = original["Password"].ToString();

// === CASE 1: Password update only ===

bool isPasswordChangeOnly = !string.IsNullOrEmpty(inputPassword)

    && inputPassword != originalPassword

    && AllOtherFieldsEmpty();

if (isPasswordChangeOnly)

{

    SqlCommand updatePwdCmd = new SqlCommand("UPDATE Customer SET Password = @Password WHERE Email = @Email", con);

    updatePwdCmd.Parameters.AddWithValue("@Password", inputPassword);

    updatePwdCmd.Parameters.AddWithValue("@Email", email);

    updatePwdCmd.ExecuteNonQuery();

```

```

        MessageBox.Show("Password updated successfully.", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);

        return;
    }

    // === CASE 2: Update other fields ===

    if (string.IsNullOrEmpty(inputPassword) || inputPassword != originalPassword)
    {

        MessageBox.Show("To update personal info, please enter the correct current password.", "Authentication Failed",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    // Get updated values (or fallback to original)

    string newPassword = (txtPassword.Text == originalPassword || string.IsNullOrEmpty(txtPassword.Text)) ? originalPassword :
txtPassword.Text;

    string newFirstName = string.IsNullOrEmpty(txtFirstName.Text) ? original["FirstName"].ToString() : txtFirstName.Text;

    string newLastName = string.IsNullOrEmpty(txtLastName.Text) ? original["LastName"].ToString() : txtLastName.Text;

    string newGovernment = string.IsNullOrEmpty(txtGovernment.Text) ? original["Government"].ToString() : txtGovernment.Text;

    string newCity = string.IsNullOrEmpty(txtCity.Text) ? original["City"].ToString() : txtCity.Text;

    string newStreet = string.IsNullOrEmpty(txtStreet.Text) ? original["Street"].ToString() : txtStreet.Text;

    string newDistrict = string.IsNullOrEmpty(txtDistrict.Text) ? original["District"].ToString() : txtDistrict.Text;

    if (MessageBox.Show("Confirm update?", "Confirm", MessageBoxButtons.YesNo, MessageBoxIcon.Question) != DialogResult.Yes)

        return;

    SqlCommand updateCmd = new SqlCommand(

        "UPDATE Customer SET Password=@Password, FirstName=@FirstName, LastName=@LastName, Government=@Government,
City=@City, Street=@Street, District=@District WHERE CID=@CID", con);

    updateCmd.Parameters.AddWithValue("@Password", newPassword);

    updateCmd.Parameters.AddWithValue("@FirstName", newFirstName);

    updateCmd.Parameters.AddWithValue("@LastName", newLastName);

    updateCmd.Parameters.AddWithValue("@Government", newGovernment);

```

```

updateCmd.Parameters.AddWithValue("@City", newCity);

updateCmd.Parameters.AddWithValue("@Street", newStreet);

updateCmd.Parameters.AddWithValue("@District", newDistrict);

updateCmd.Parameters.AddWithValue("@CID", customerId);

updateCmd.ExecuteNonQuery();


// Update phone numbers

bool hasPhone1 = !string.IsNullOrEmpty(txtPhoneNo1.Text);

bool hasPhone2 = !string.IsNullOrEmpty(txtPhoneNo2.Text);


if (hasPhone1 || hasPhone2)

{

    new SqlCommand("DELETE FROM CustomerPhone WHERE CID=@CID", con) { Parameters = { new SqlParameter("@CID", customerId)
} }.ExecuteNonQuery();


    SqlCommand insertPhone = new SqlCommand("INSERT INTO CustomerPhone (CID, PhoneNo) VALUES (@CID, @PhoneNo)", con);

    insertPhone.Parameters.Add("@CID", SqlDbType.Int).Value = customerId;

    insertPhone.Parameters.Add("@PhoneNo", SqlDbType.VarChar);


    if (hasPhone1)

    {

        insertPhone.Parameters["@PhoneNo"].Value = txtPhoneNo1.Text.Trim();

        insertPhone.ExecuteNonQuery();

    }


    if (hasPhone2)

    {

        insertPhone.Parameters["@PhoneNo"].Value = txtPhoneNo2.Text.Trim();

        insertPhone.ExecuteNonQuery();

    }

}

```

```

        MessageBox.Show("Customer updated successfully!", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);

        btnShow_Click(null, null);

    }

}

catch (Exception ex)

{

    MessageBox.Show("Error: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

}

}

private bool AllOtherFieldsEmpty()

{

    return string.IsNullOrEmpty(txtFirstName.Text)

        && string.IsNullOrEmpty(txtLastName.Text)

        && string.IsNullOrEmpty(txtGovernment.Text)

        && string.IsNullOrEmpty(txtCity.Text)

        && string.IsNullOrEmpty(txtStreet.Text)

        && string.IsNullOrEmpty(txtDistrict.Text)

        && string.IsNullOrEmpty(txtPhoneNo1.Text)

        && string.IsNullOrEmpty(txtPhoneNo2.Text);

}

}

}

```

-The show data function works as follows:

For the customer table the button is used in a simpler way by just showing all the data related to all customers once it is clicked and this is done by running this code:-

```

private void btnShow_Click(object sender, EventArgs e)

{

    try

```

```

{

    using (SqlConnection con = new SqlConnection(@"Data Source=.\SQLEXPRESS;Initial Catalog=app;Integrated
Security=True"))

    {

        con.Open();

        // Load Customer data

        using (SqlDataAdapter customerAdapter = new SqlDataAdapter("SELECT * FROM Customer", con))

        {

            DataTable customerTable = new DataTable();

            customerAdapter.Fill(customerTable);

            dataGridView2.DataSource = customerTable;

        }

        // Load CustomerPhone data

        using (SqlDataAdapter phoneAdapter = new SqlDataAdapter("SELECT * FROM CustomerPhone", con))

        {

            DataTable phoneTable = new DataTable();

            phoneAdapter.Fill(phoneTable);

            dgvCustomer2.DataSource = phoneTable;

        }

    }

    catch (Exception ex)

    {

        MessageBox.Show("Error loading data: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

    }

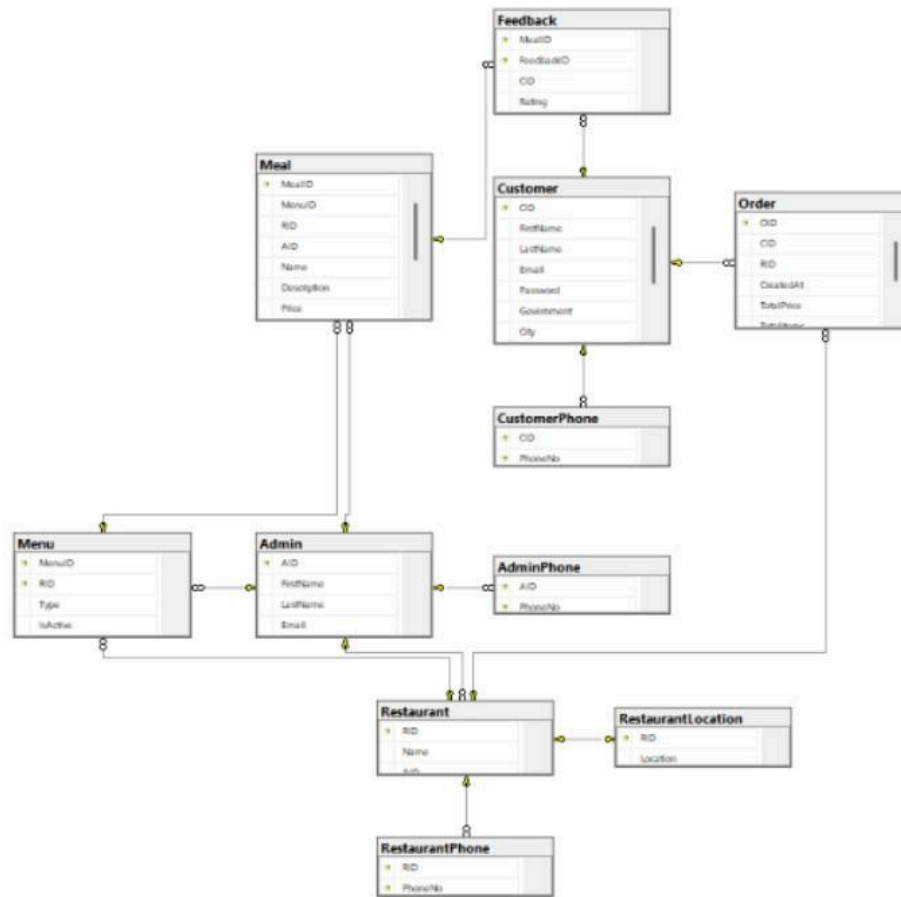
}

```

PowerDesigner [CDM Conceptual Data_1, Diagram_1 - C:\Users\dell\OneDrive\Desktop\Food ordering system2.cdm]



Database Diagram in SQL:



SQL:

-- Food Ordering System Database

CREATE DATABASE FoodOrderingSystem;

GO

USE FoodOrderingSystem;

GO

-- CUSTOMER TABLE

CREATE TABLE Customer (

 CID INT PRIMARY KEY,

 FirstName VARCHAR(50) NOT NULL,

 LastName VARCHAR(50) NOT NULL,

 Email VARCHAR(100) NOT NULL,

 Password VARCHAR(100) NOT NULL,

 Government VARCHAR(50) NOT NULL,

 City VARCHAR(50) NOT NULL,

 District VARCHAR(50) NOT NULL,

 Street VARCHAR(50) NOT NULL

);

CREATE TABLE CustomerPhone (

 CID INT NOT NULL,

 PhoneNo VARCHAR(11) NOT NULL,

 PRIMARY KEY (CID, PhoneNo),

 FOREIGN KEY (CID) REFERENCES Customer(CID) ON DELETE CASCADE

);

-- ADMIN TABLE

```
CREATE TABLE Admin (  
  
    AID INT PRIMARY KEY,  
  
    FirstName VARCHAR(50) NOT NULL,  
  
    LastName VARCHAR(50) NOT NULL,  
  
    Email VARCHAR(100) NOT NULL,  
  
    Password VARCHAR(100) NOT NULL  
  
);
```

```
CREATE TABLE AdminPhone (  
  
    AID INT NOT NULL,  
  
    PhoneNo VARCHAR(11) NOT NULL,  
  
    PRIMARY KEY (AID, PhoneNo),  
  
    FOREIGN KEY (AID) REFERENCES Admin(AID) ON DELETE CASCADE  
  
);
```

-- RESTAURANT TABLE

```
CREATE TABLE Restaurant (  
  
    RID INT PRIMARY KEY,  
  
    Name VARCHAR(100) NOT NULL,  
  
    AID INT,  
  
    FOREIGN KEY (AID) REFERENCES Admin(AID) ON DELETE SET NULL  
  
);
```

```
CREATE TABLE RestaurantPhone (  
  
    RID INT NOT NULL,  
  
    PhoneNo VARCHAR(11) NOT NULL,  
  
    PRIMARY KEY (RID, PhoneNo),  
  
    FOREIGN KEY (RID) REFERENCES Restaurant(RID) ON DELETE CASCADE
```

);

```
CREATE TABLE RestaurantLocation (  
  
    RID INT PRIMARY KEY,  
  
    Location VARCHAR(100) NOT NULL,  
  
    FOREIGN KEY (RID) REFERENCES Restaurant(RID) ON DELETE CASCADE  
  
);
```

-- MENU TABLE (weak entity)

```
CREATE TABLE Menu (  
  
    MenuID INT NOT NULL,  
  
    RID INT NOT NULL,  
  
    Type VARCHAR(50) NOT NULL,  
  
    IsActive BIT NOT NULL,  
  
    AID INT,  
  
    PRIMARY KEY (MenuID, RID),  
  
    FOREIGN KEY (RID) REFERENCES Restaurant(RID) ON DELETE CASCADE,  
  
    FOREIGN KEY (AID) REFERENCES Admin(AID) ON DELETE SET NULL  
  
);
```

-- MEAL TABLE

```
CREATE TABLE Meal (  
  
    MealID INT PRIMARY KEY,  
  
    MenuID INT NOT NULL,  
  
    RID INT NOT NULL,  
  
    AID INT,  
  
    Name VARCHAR(100) NOT NULL,  
  
    Description VARCHAR(500),  
  
    Price DECIMAL(10,2) NOT NULL,
```

```
IsAvailable BIT NOT NULL,  
  
Category VARCHAR(50),  
  
FOREIGN KEY (MenuID, RID) REFERENCES Menu(MenuID, RID) ON DELETE CASCADE,  
  
FOREIGN KEY (AID) REFERENCES Admin(AID) ON DELETE SET NULL  
  
);
```

-- FEEDBACK TABLE (weak entity)

```
CREATE TABLE Feedback (  
  
    MealID INT NOT NULL,  
  
    FeedbackID INT NOT NULL,  
  
    CID INT NULL,  
  
    Rating INT NOT NULL CHECK (Rating BETWEEN 1 AND 5),  
  
    Comment VARCHAR(MAX),  
  
    PRIMARY KEY (MealID, FeedbackID),  
  
    FOREIGN KEY (MealID) REFERENCES Meal(MealID) ON DELETE CASCADE,  
  
    FOREIGN KEY (CID) REFERENCES Customer(CID) ON DELETE SET NULL  
  
);
```

-- ORDER TABLE

```
CREATE TABLE [Order] (  
  
    OID INT PRIMARY KEY,  
  
    CID INT NOT NULL,  
  
    RID INT NOT NULL,  
  
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),  
  
    TotalPrice DECIMAL(10, 2) NOT NULL,  
  
    TotalItems INT NOT NULL,  
  
    PaymentMethod VARCHAR(50) NOT NULL,  
  
    FOREIGN KEY (CID) REFERENCES Customer(CID) ON DELETE CASCADE,  
  
    FOREIGN KEY (RID) REFERENCES Restaurant(RID) ON DELETE CASCADE
```

);

-- RELATIONSHIP TABLES:

-- ORDERDETAILS TABLE

```
CREATE TABLE OrderMeal (  
    OID INT NOT NULL,  
    MealID INT NOT NULL,  
    Quantity INT NOT NULL,  
    Price DECIMAL(10, 2) NOT NULL,  
    PRIMARY KEY (OID, MealID),  
    FOREIGN KEY (OID) REFERENCES [Order](OID) ON DELETE CASCADE,  
    FOREIGN KEY (MealID) REFERENCES Meal(MealID) ON DELETE CASCADE  
);
```

```
CREATE TABLE AdminManagesCustomer (  
    AID INT NOT NULL,  
    CID INT NOT NULL,  
    ManagedAt DATETIME DEFAULT GETDATE(),  
    PRIMARY KEY (AID, CID),  
    FOREIGN KEY (AID) REFERENCES Admin(AID) ON DELETE CASCADE,  
    FOREIGN KEY (CID) REFERENCES Customer(CID) ON DELETE CASCADE  
);
```

```
CREATE TABLE CustomerViewsMenu (  
    CID INT NOT NULL,  
    MenuID INT NOT NULL,  
    RID INT NOT NULL,  
    ViewedAt DATETIME DEFAULT GETDATE(),
```

```
PRIMARY KEY (CID, MenuID, RID),  
  
FOREIGN KEY (CID) REFERENCES Customer(CID) ON DELETE CASCADE,  
  
FOREIGN KEY (MenuID, RID) REFERENCES Menu(MenuID, RID) ON DELETE CASCADE  
);
```

```
CREATE TABLE RestaurantPreparesOrder (  
  
    RID INT NOT NULL,  
  
    OID INT NOT NULL,  
  
    Status VARCHAR(50),  
  
    PRIMARY KEY (RID, OID),  
  
    FOREIGN KEY (RID) REFERENCES Restaurant(RID) ON DELETE CASCADE,  
  
    FOREIGN KEY (OID) REFERENCES [Order](OID) ON DELETE CASCADE  
);
```

```
CREATE TABLE AdminManagesRestaurant (  
  
    AID INT NOT NULL,  
  
    RID INT NOT NULL,  
  
    PRIMARY KEY (AID, RID),  
  
    FOREIGN KEY (AID) REFERENCES Admin(AID) ON DELETE CASCADE,  
  
    FOREIGN KEY (RID) REFERENCES Restaurant(RID) ON DELETE CASCADE  
);
```

```
CREATE TABLE AdminManagesMenu (  
  
    AID INT NOT NULL,  
  
    MenuID INT NOT NULL,  
  
    RID INT NOT NULL,  
  
    PRIMARY KEY (AID, MenuID, RID),  
  
    FOREIGN KEY (AID) REFERENCES Admin(AID) ON DELETE CASCADE,  
  
    FOREIGN KEY (MenuID, RID) REFERENCES Menu(MenuID, RID) ON DELETE CASCADE
```

);

CREATE TABLE AdminManagesMeal (

AID INT NOT NULL,

MealID INT NOT NULL,

PRIMARY KEY (AID, MealID),

FOREIGN KEY (AID) REFERENCES Admin(AID) ON DELETE CASCADE,

FOREIGN KEY (MealID) REFERENCES Meal(MealID) ON DELETE CASCADE

);

CREATE TABLE CustomerControlsFeedback (

CID INT NOT NULL,

FeedbackID INT NOT NULL,

MealID INT NOT NULL,

PRIMARY KEY (CID, FeedbackID, MealID),

FOREIGN KEY (CID) REFERENCES Customer(CID) ON DELETE CASCADE,

FOREIGN KEY (FeedbackID, MealID) REFERENCES Feedback(FeedbackID, MealID) ON DELETE CASCADE

);

CREATE TABLE MealReceivesFeedback (

MealID INT NOT NULL,

FeedbackID INT NOT NULL,

PRIMARY KEY (MealID, FeedbackID),

FOREIGN KEY (MealID) REFERENCES Meal(MealID) ON DELETE CASCADE,

FOREIGN KEY (FeedbackID) REFERENCES Feedback(FeedbackID) ON DELETE CASCADE

);

-- Customer → Order already handled by [Order].CID FK

-- Restaurant → Menu already handled by Menu.RID FK (weak entity)

-- Menu → Meal already handled by Meal(MenuID, RID) FK to Menu(MenuID, RID)

questions:

--A) what was the most ordering meal?

```
SELECT TOP 1 M.MealID, M.Name, COUNT(*) AS OrderCount
FROM OrderMeal OM
JOIN Meal M ON OM.MealID = M.MealID
GROUP BY M.MealID, M.Name
ORDER BY OrderCount DESC;
```

--B) What were the order prices for each customer during the last three months?

```
SELECT C.CID, C.FirstName, C.LastName, O.TotalPrice, O.CreatedAt
FROM [Order] O
JOIN Customer C ON O.CID = C.CID
WHERE O.CreatedAt >= DATEADD(MONTH, -3, GETDATE());
```

--C) What was the list of meals that were not ordered by any customer?

```
SELECT MealID, Name
FROM Meal
WHERE MealID NOT IN (
    SELECT DISTINCT MealID
    FROM OrderMeal
);
```

--D) Who was the customer that made the highest order price this month?

```
SELECT TOP 1 C.CID, C.FirstName, C.LastName, O.TotalPrice
FROM [Order] O
JOIN Customer C ON O.CID = C.CID
WHERE MONTH(O.CreatedAt) = MONTH(GETDATE()) AND YEAR(O.CreatedAt) = YEAR(GETDATE())
ORDER BY O.TotalPrice DESC;
```

--E) What was the list of meals that were ordered more than five times during the last two months?

```
SELECT M.MealID, M.Name, SUM(OM.Quantity) AS TotalOrdered
FROM OrderMeal OM
JOIN Meal M ON OM.MealID = M.MealID
JOIN [Order] O ON OM.OID = O.OID
WHERE O.CreatedAt >= DATEADD(MONTH, -2, GETDATE())
GROUP BY M.MealID, M.Name
HAVING SUM(OM.Quantity) > 5;
```

--F) For each customer, retrieve all his/her information and the number of orders

```
SELECT C.*, COUNT(O.OID) AS OrderCount
FROM Customer C
LEFT JOIN [Order] O ON C.CID = O.CID
GROUP BY C.CID, C.FirstName, C.LastName, C.Email, C.Password, C.Government, C.City, C.District, C.Street;
```